



10.3. Double-Ended Queue

Previous Sections:

 [10. Queues](#)

 [10.1. Operations on Queues](#)

 [10.2. Circular Queues](#)

So far, we have studied two types of queues:

- **Linear Queue**, where insertion occurs only at the rear and deletion occurs only at the front.
- **Circular Queue**, which improves memory utilization by allowing the queue to wrap around and reuse previously freed locations.

However, both structures still follow a strict rule: elements enter from one end and leave from the other. What if we want more flexibility?

Suppose we want to insert an urgent item directly at the front of the queue, or remove an item from the rear instead of the front. Traditional queues do not allow such operations.

To overcome this limitation, we introduce the **Double-Ended Queue**, commonly known as a **Deque** (it is pronounced "*deck*").

A Deque is a special type of queue in which both insertion and deletion can occur at either end.

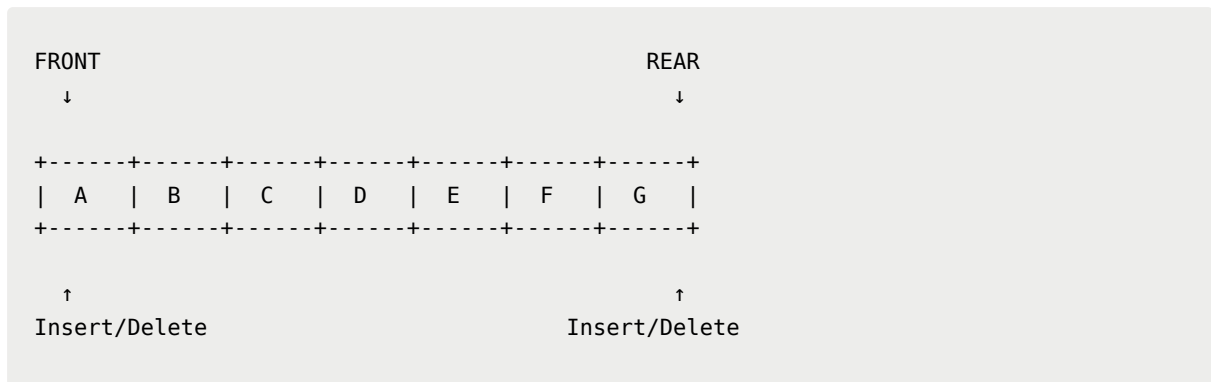
1. What is a Deque?

A **Double-Ended Queue (Deque)** is a queue that supports operations at both the FRONT and the REAR.

Unlike ordinary queues:

- Elements can be inserted from the front.
- Elements can be inserted from the rear.
- Elements can be removed from the front.
- Elements can be removed from the rear.

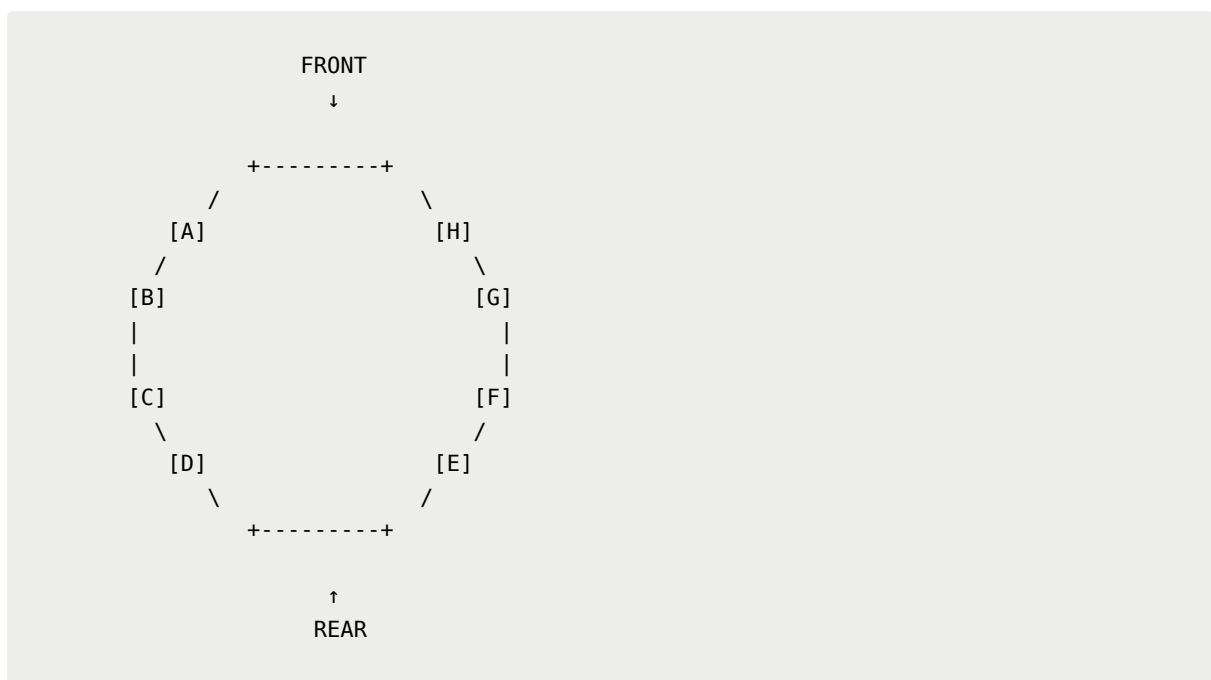
A deque can be visualized as follows:



Both ends are accessible for insertion and deletion.

2. Deque as a Circular Structure

Like Circular Queues, deques are often implemented using circular structures to efficiently utilize memory. A circular deque may be visualized as:



In this structure:

- Moving past the last position returns to the beginning.
- Both FRONT and REAR can wrap around.
- Memory is reused efficiently.

Many real-world deque implementations are built using circular arrays or doubly linked lists.

3. Basic Operations of a Deque

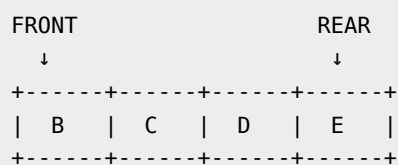
A deque supports four primary operations.

Operation	Description	Time Complexity
ENQUEUE FRONT	Insert an element at the front	$O(1)$
ENQUEUE REAR	Insert an element at the rear	$O(1)$
DEQUEUE FRONT	Remove an element from the front	$O(1)$
DEQUEUE REAR	Remove an element from the rear	$O(1)$

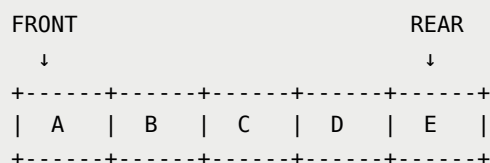
ENQUEUE FRONT

This operation inserts a new element before the current FRONT.

Before insertion:



Insert A at the front:



The FRONT moves one position toward the newly inserted element.

ENQUEUE REAR

```

Before insertion:

FRONT                                REAR
↓                                  ↓
+-----+-----+-----+-----+
|  A   |  B   |  C   |  D   |
+-----+-----+-----+-----+

Insert E:

FRONT                                REAR
↓                                  ↓
+-----+-----+-----+-----+
|  A   |  B   |  C   |  D   |  E   |
+-----+-----+-----+-----+

```

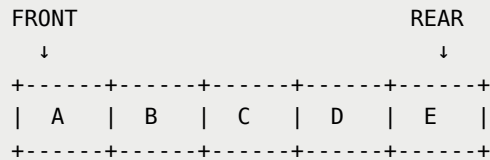
DEQUEUE FRONT

[illegible]

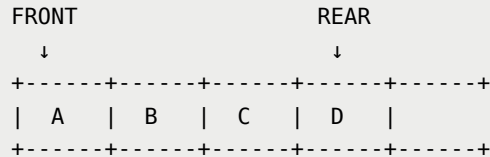
DEQUEUE REAR

10.3. Double-Ended Queue

Before deletion:



Remove E:



The REAR moves toward the previous element.

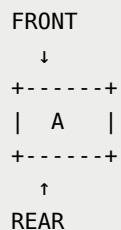
Special Cases

- Empty Deque: Both FRONT and REAR point to the same element. This is true whether we use: Enqueue Front and Enqueue Rear for the first insertion.

```
FRONT = None
REAR  = None
```

```
[ EMPTY ]
```

When the first element is inserted:



- Single Element Deque: A deque containing one element. The deque returns to its empty state.

```
FRONT
  ↓
```

```

+-----+
|  A  |
+-----+
  ↑
 REAR

```

Removing that element causes:

```

FRONT = None
REAR  = None

```

Why Use a Doubly Linked List?

A deque requires efficient movement in both directions. A singly linked list only provides access to the next node: `A → B → C → D`. Moving backward is impossible.

A doubly linked list provides both next and previous references: `A ⇄ B ⇄ C ⇄ D`

This allows: fast insertion at the front; fast insertion at the rear; fast deletion at the front; fast deletion at the rear. All in $O(1)$ time.

For this reason, doubly linked lists are one of the most common implementations of deques.

4. Python Implementation Using Doubly Linked Lists

- Node Class

```

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None

```

- Deque Class

```

class Deque:
    def __init__(self):
        self.front = None
        self.rear = None
        self.size = 0

```

```

def is_empty(self):
    return self.size == 0

def enqueue_front(self, data):
    new_node = Node(data)

    if self.is_empty():
        self.front = self.rear = new_node
    else:
        new_node.next = self.front
        self.front.prev = new_node
        self.front = new_node

    self.size += 1

def enqueue_rear(self, data):
    new_node = Node(data)

    if self.is_empty():
        self.front = self.rear = new_node
    else:
        new_node.prev = self.rear
        self.rear.next = new_node
        self.rear = new_node

    self.size += 1

def dequeue_front(self):
    if self.is_empty():
        print("Deque is empty")
        return None

    removed_data = self.front.data

    if self.front == self.rear:
        self.front = self.rear = None
    else:

```

```

        self.front = self.front.next
        self.front.prev = None

    self.size -= 1
    return removed_data

def dequeue_rear(self):
    if self.is_empty():
        print("Deque is empty")
        return None

    removed_data = self.rear.data

    if self.front == self.rear:
        self.front = self.rear = None
    else:
        self.rear = self.rear.prev
        self.rear.next = None

    self.size -= 1
    return removed_data

def traverse(self):
    if self.is_empty():
        print("Deque is empty")
        return

    current = self.front

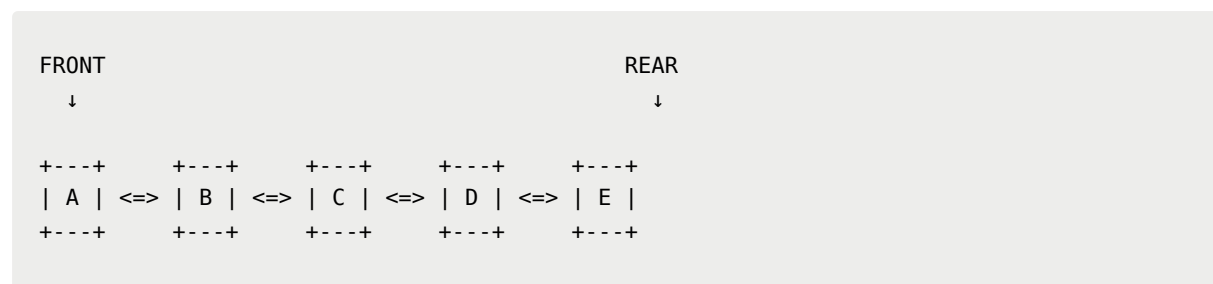
    while current:
        print(current.data, end=" <=> ")
        current = current.next

    print("None")

```

5. Visualizing the Doubly Linked List Deque

After inserting several elements:



Each node can move forward using next or backward using prev. This makes front and rear operations equally efficient.

6. Advantages of Deques

- Insertion can occur at both ends.
- Deletion can occur at both ends.
- Very flexible compared to ordinary queues.
- Efficient implementation using doubly linked lists.
- Frequently used in caching, scheduling, and sliding window algorithms.

7. Disadvantages of Deques

- More complex than ordinary queues.
- Requires additional memory for previous pointers in doubly linked lists.
- Harder to implement correctly than linear queues.

Summary

A **Double-Ended Queue (Deque)** is a special queue that allows insertion and deletion at both the FRONT and REAR.

Unlike ordinary queues that strictly follow one insertion point and one deletion point, deques provide greater flexibility while maintaining efficient $O(1)$ operations at both ends.

To support bidirectional operations efficiently, deques are commonly implemented using **Doubly Linked Lists**, where each node stores references to

both the next and previous nodes.

The four fundamental operations are:

- Enqueue Front
- Enqueue Rear
- Dequeue Front
- Dequeue Rear

Because of their flexibility and efficiency, dequeues are widely used in operating systems, caching systems, scheduling algorithms, and many advanced data-processing applications.

References

https://drive.google.com/file/d/1A7WberDJxGmGtXpGZ7v-FFATGRlgKoQZ/view?usp=drive_link

<https://www.programiz.com/dsa/dequeue>

<https://www.geeksforgeeks.org/dsa/dequeue-set-1-introduction-applications/>

Next Section:

 [11. Trees](#)